TALLER PRÁCTICO COLABORATIVO

Git y GitHub: Ramas, Repositorio Remoto, Push, Pull,
Resolución de Conflictos y .gitignore

Docente: Jhonathan Arley Peñaloza Sierra

Integrantes del equipo:	Dilan Fonseca,Deyvid Godoy ,Breinner lopez
Fecha:	04/MAYO/2026

Objetivo de la actividad

Simular un proyecto de desarrollo colaborativo real donde los estudiantes trabajan en equipo usando Git y GitHub. Practicarán la gestión de ramas, la sincronización con repositorios remotos (push/pull), la resolución de conflictos de merge y el uso de .gitignore. Al finalizar, cada equipo habrá experimentado y resuelto los problemas más comunes que enfrentan los desarrolladores en su día a día.

Requisitos previos

Tener Git instalado y configurado (user.name, user.email).

Tener una cuenta en GitHub.

Tener Visual Studio Code instalado.

Haber completado los módulos anteriores: Introducción a Git (repositorio local, commits, staging area).

Organización del equipo

Formen equipos de 3 personas. Cada integrante trabajará desde su propio computador con su propia cuenta de GitHub. Un integrante será quien cree el repositorio inicial, pero TODOS deberán clonar, crear ramas, hacer push, pull y resolver conflictos. Asegurarse de que las 3 personas estén sentadas cerca para coordinarse.

Entregables esperados

Repositorio en GitHub con al menos 3 ramas fusionadas en main.

Historial de commits usando Conventional Commits.

Archivo .gitignore configurado correctamente.

Este documento completo con capturas de pantalla y respuestas.

Al menos 1 conflicto resuelto documentado (captura de pantalla del conflicto y la solución).

Parte 1: Configuración del Proyecto Colaborativo

Ejercicio 1.1 – Integrante A: Crear el repositorio

Solo UN integrante del equipo (Integrante A) realiza estos pasos. Los demás observan.

Paso 1: Crear el proyecto local

Integrante A crea una carpeta llamada "proyecto-equipo" y la abre en VS Code.

Paso 2: Inicializar Git y crear archivos

```
# Inicializar Git
git init

# Crear archivos del proyecto
```

Crear los siguientes archivos con el contenido indicado: no necesariamente deben crear los archivos como se muestran pueden personalizar con otros tipos de archivo.

Archivo	Contenido
README.md	`# Proyecto Equipo` `Proyecto colaborativo para practicar Git y control de versiones.`
styles.css	`body { font-family: Arial; margin: 0; padding: 20px; }`
README.md	`# Proyecto Equipo\nProyecto colaborativo para practicar Git.`
.gitignore	`node_modules/\n.env\n*.log`

Paso 3: Primer commit

```
git add .
git commit -m 'feat: crear estructura inicial del proyecto'
```

Paso 4: Crear repositorio en GitHub y enlazar

Integrante A crea un repositorio público en GitHub llamado "proyecto-equipo" y ejecuta:

```
git remote add origin https://github.com/USUARIO/proyecto-equipo.git
git branch -M main
git push -u origin main
```

a) Peguen aquí la URL del repositorio creado en GitHub:

https://github.com/Dafcgg/proyecto-equipo.git

Ejercicio 1.2 – Integrantes B y C: Clonar el repositorio

Los otros 2 integrantes clonan el repositorio en sus computadores:

```
git clone https://github.com/USUARIO/proyecto-equipo.git
cd proyecto-equipo
```

b) ¿Todos los integrantes lograron clonar el repositorio? ¿Qué archivos ven al abrirlo en VS Code?

Si, todos los integrantes clonaron el repositorio en sus computadoras.los archivos que ven fueron el index.htm,README.md, styles.css.

---

c) Ejecuten git log --oneline en los 3 computadores. ¿Ven el mismo commit?

Si se ve el commit de feat:crear esctructura inicial del proyecto en los computadores.

Parte 2: Trabajo con Ramas (Cada uno en su rama)

Ejercicio 2.1 – Cada integrante crea su propia rama

CADA integrante crea una rama con su nombre y trabaja en ella desde su propio computador:

Integrante	Comando	Tarea
A	`git branch feature-headergit checkout feature-header`	Modificar el <h1> en index.html y agregar un <h2> con subtítulo
B	`git branch feature-navgit checkout feature-nav`	Agregar un menú de navegación (<nav>) en index.html debajo del <h1>
C	`git branch feature-footergit checkout feature-footer`	Agregar un pie de página (<footer>) al final del <body> en index.html

Cada integrante hace commit y push de su rama

Después de hacer los cambios, cada uno ejecuta:

```
git add .
git commit -m 'feat: agregar [lo que hiciste]'
git push -u origin [nombre-de-tu-rama]
```

a) ¿Cuántas ramas aparecen ahora en GitHub? Listar sus nombres:

Aparecen 4 ramas en GitHub:

- main
- feature-header
- feature-nav
- feature-footer

b) Ejecuten git branch en cada computador. ¿Ven las ramas de los demás? ¿Por qué?

Sí, después de ejecutar `git pull` ya es posible ver las ramas de los demás integrantes en el repositorio local. Esto se debe a que este comando no solo actualiza la rama actual, sino que también trae información del repositorio remoto.

Parte 3: Fusionar Ramas sin Conflictos

Ejercicio 3.1 – Integrante A fusiona su rama primero

Integrante A fusiona su rama feature-header en main. Como es el primero, NO habrá conflictos:

```
# Moverse a main
git checkout main

# Traer últimos cambios del remoto
git pull

# Fusionar la rama
git merge feature-header

# Subir al remoto
git push
```

a) ¿Se fusionó sin problemas? ¿Qué mensaje mostró Git?

Sí, se fusionó sin problemas. Git mostró un mensaje de tipo “fast-forward”, indicando que la rama se integró correctamente sin conflictos.

Ejercicio 3.2 – Integrante B fusiona su rama (posible conflicto)

Integrante B intenta fusionar su rama feature-nav. ANTES de hacer merge, debe traer los cambios de A:

```
git checkout main
git pull
git merge feature-nav
```

> **¿Qué pasa aquí?**
> Como tanto A como B modificaron index.html, es MUY PROBABLE que Git genere un conflicto. Si no hay conflicto, Git fusionó automáticamente porque los cambios estaban en zonas diferentes del archivo. Si hay conflicto, pasar a la Parte 4.

b) ¿Hubo conflicto o no? Describan qué pasó:

Sí hubo conflicto. Git mostró un mensaje indicando un conflicto en el archivo `index.html` porque ambos cambios afectaban el mismo archivo. Fue necesario revisar el archivo y resolver el conflicto manualmente.

Parte 4: Provocar y Resolver Conflictos Intencionalmente

> **Objetivo de esta parte**
> Vamos a provocar un conflicto A PROPÓSITO para que aprendan a resolverlo. Esto es algo que pasará constantemente en proyectos reales.

Ejercicio 4.1 – Provocar un conflicto

Los integrantes A y B van a modificar EXACTAMENTE la misma línea del mismo archivo:

Integrante A (desde main):

```
# Asegurarse de estar en main actualizado
git checkout main
git pull
```

```
# Modificar la línea del <h1> en index.html
# Cambiar: <h1>Bienvenidos</h1>
# Por:    <h1>Bienvenidos al Proyecto - Versión A</h1>

git add index.html
git commit -m 'refactor: actualizar título principal'
git push
```

Integrante B (desde main):

```
# Asegurarse de estar en main (SIN hacer pull)
git checkout main

# Modificar la MISMA línea del <h1> en index.html
# Cambiar: <h1>Bienvenidos</h1>
# Por:    <h1>Bienvenidos al Proyecto - Versión B</h1>

git add index.html
git commit -m 'refactor: actualizar título principal'

# Intentar hacer push
git push
```

> **¡ERROR! Git rechazará el push**
> Git mostrará un error porque el remoto tiene cambios que tú no tienes. El mensaje dirá algo como: "Updates were rejected because the remote contains work that you do not have locally." La solución es hacer git pull primero.

a) ¿Qué mensaje de error les mostró Git al hacer push? Copíenlo aquí:

! [rejected] main -> main (fetch first)
error: falló el empuje de algunas referencias a 'https://github.com/Dafcgg/proyecto-equipo.git'

Ejercicio 4.2 – Resolver el conflicto

Integrante B ahora debe hacer pull para traer los cambios de A:

```
git pull
```

CONFLICTO DE MERGE

Git mostrará un mensaje indicando que hay un conflicto en index.html. Al abrir el archivo, verán marcadores de conflicto como estos:

```
<<<<<<< HEAD
<h1>Bienvenidos al Proyecto - Versión B</h1>
=======
<h1>Bienvenidos al Proyecto - Versión A</h1>
>>>>>>> origin/main
```

VS Code mostrará opciones para resolver el conflicto:

Opción	Qué hace
Accept Current	Se queda con TU versión (Versión B) - lo que tenías antes del pull
Accept Incoming	Se queda con la versión del REMOTO (Versión A) - lo que subió el Integrante A
Accept Both	Conserva AMBAS versiones una debajo de la otra

Elijan la opción que prefieran (o editen manualmente) y luego:

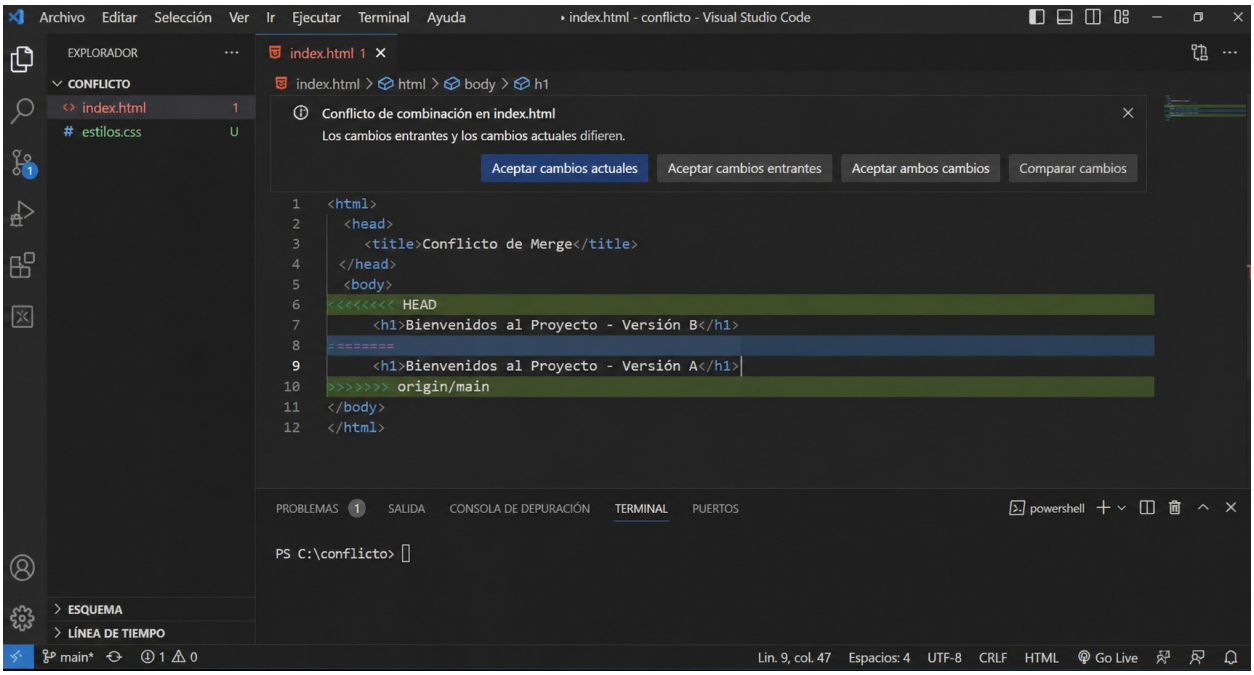
```
# Después de resolver el conflicto
git add index.html
git commit -m 'fix: resolver conflicto en título principal'
git push
```

b) ¿Qué opción eligieron para resolver el conflicto? ¿Por qué?

Accept Incoming
para quedarnos con la version A.

---

c) Peguen una captura de pantalla del conflicto tal como apareció en VS Code:



---

d) Ahora Integrante C hace git pull. ¿Ve los cambios resueltos? Confirmen:

Sí, el integrante C verá los cambios ya resueltos. Al hacer `git pull`, descargará la versión final del archivo `index.html` sin conflictos, ya que el conflicto fue solucionado previamente y los cambios fueron confirmados (`commit`) y subidos al repositorio.

Parte 5: Configuración del .gitignore

Ejercicio 5.1 – Probar el .gitignore

Cualquier integrante crea los siguientes archivos en el proyecto (son archivos que NO deben subirse):

```
# Crear archivos que deben ser ignorados
# (Crear manualmente o por terminal)

# Archivo con contraseñas (NUNCA subir)
# .env con contenido: PASSWORD=miContraseña123

# Archivo de log temporal
```

```
# debug.log con contenido: [ERROR] algo falló
```

a) Ejecuten git status. ¿Aparecen .env y debug.log como archivos para rastrear? ¿Por qué no?

Al ejecutar git status, los archivos .env y debug.log no aparecen como archivos para rastrear. Esto ocurre porque desde el inicio del proyecto se configuró el archivo .gitignore, donde se incluyeron reglas como:

env → para ignorar archivos con variables sensibles
*.log → para ignorar archivos de registro

Gracias a esta configuración, Git reconoce que esos archivos no deben ser versionados ni subidos al repositorio, por lo que los excluye automáticamente del seguimiento.

> **Respuesta esperada**
> NO aparecen porque ya están en el .gitignore que se creó al inicio. El .gitignore contiene .env y *.log, por lo que Git los ignora automáticamente.

b) ¿Qué pasaría si no tuviéramos el .gitignore y subiéramos el archivo .env a GitHub?

Si no tuviéramos .gitignore y subimos el archivo .env a GitHub, estaríamos exponiendo información sensible como contraseñas o claves. Esto podría permitir accesos no autorizados y generar problemas de seguridad en el proyecto.

Parte 6: Guía de Referencia – Problemas Comunes y Soluciones

Lean esta guía en equipo. Les servirá como referencia para resolver problemas en futuros proyectos.

> **Problema 1: Push rechazado**
> Error: "Updates were rejected because the remote contains work that you do not have locally."
> Causa: Alguien subió cambios al remoto que tú no tienes en tu local.

Solución: Hacer git pull antes de git push. Si hay conflictos, resolverlos.

```
# Solución
git pull
# Resolver conflictos si los hay
git add .
git commit -m 'fix: resolver conflicto'
git push
```

Problema 2: Conflicto de merge

Error: "CONFLICT (content): Merge conflict in archivo.html"
Causa: Dos personas modificaron la misma línea del mismo archivo.
Solución: Abrir el archivo, elegir Current/Incoming/Both, guardar, add y commit.

```
# Después de resolver manualmente
git add archivo-con-conflicto.html
git commit -m 'fix: resolver conflicto en archivo'
git push
```

Problema 3: Rama desactualizada

Síntoma: Tu rama de trabajo está muy atrás de main.
Causa: No actualizaste tu rama con los últimos cambios de main.
Solución: Desde tu rama, hacer merge de main para actualizarla ANTES del merge final.

```
# Desde tu rama de trabajo
git checkout mi-rama
git merge main
# Resolver conflictos si los hay
# Luego sí, hacer el merge final en main
```

Problema 4: No puedo cambiar de rama

Error: "Your local changes would be overwritten by checkout."
Causa: Tienes archivos modificados que no has guardado con commit.
Solución: Hacer commit de tus cambios o descartarlos antes de cambiar.

```
# Opción 1: Guardar cambios
git add .
```

```
git commit -m 'feat: guardar trabajo en progreso'
git checkout otra-rama

# Opción 2: Descartar cambios (CUIDADO, se pierden)
git checkout -- .
git checkout otra-rama
```

Parte 7: Buenas Prácticas para Evitar Conflictos

#	Buena Práctica
1	Hacer git pull SIEMPRE antes de empezar a trabajar cada día
2	Trabajar cada persona en su propia rama (NUNCA directamente en main)
3	Hacer commits pequeños y frecuentes con mensajes descriptivos (Conventional Commits)
4	Antes de hacer merge a main, actualizar tu rama con los últimos cambios de main
5	Comunicarse con el equipo: avisar cuando se va a hacer push o merge
6	Configurar .gitignore DESDE EL INICIO del proyecto
7	Eliminar ramas después de fusionarlas para mantener el repositorio organizado
8	No modificar la misma línea del mismo archivo que otro compañero al mismo tiempo

Parte 8: Cheat Sheet – Comandos Utilizados

Completen esta tabla con la descripción de cada comando que usaron durante el taller:

Comando	¿Qué hace?
`git clone URL`	Descarga una copia de un repositorio remoto a tu equipo local.
`git branch nombre`	Crea una nueva rama con el nombre que elijas.
`git checkout nombre`	Te cambia de la rama actual a la rama especificada.
`git merge nombre`	Une los cambios de la rama especificada con tu rama actual.
`git branch -D nombre`	Fuerza el borrado de una rama, incluso si tiene cambios sin fusionar.
`git branch -m nuevo`	Cambia el nombre de la rama en la que te encuentras actualmente.

`git remote add origin URL`	Conecta tu repositorio local con un servidor remoto por primera vez.
`git push`	Sube tus cambios locales confirmados al repositorio remoto.
`git pull`	Descarga y fusiona los cambios del servidor remoto en tu rama local.
`git push -u origin rama`	Sube la rama y la vincula al remoto para futuros "push/pull" simples.
`git log --oneline`	Muestra el historial de commits resumido en una sola línea cada uno.

Parte 9: Reflexión Final

a) ¿Cuál fue el momento más difícil del taller? ¿Cómo lo resolvieron?

al hacer *merge* entre ramas, porque no entendíamos bien las diferencias entre cambios. Lo solucionamos revisando el archivo en conflicto, comparando ambas versiones, eligiendo los cambios correctos, eliminando las marcas de conflicto y haciendo un nuevo commit.

---

b) ¿Por qué es importante hacer git pull antes de empezar a trabajar?

Es importante hacer `git pull` porque así obtenemos la versión más actualizada del repositorio. Esto evita trabajar sobre archivos desactualizados y reduce la probabilidad de conflictos cuando hagamos *push*. Además, permite sincronizar el trabajo de todo el equipo desde el inicio.

---

c) ¿Qué pasaría en un proyecto real si NO se usa .gitignore y se sube un archivo .env con contraseñas?

Sería un problema grave de seguridad. El archivo `.env` puede contener contraseñas, claves de API y datos sensibles. Si se sube al repositorio, especialmente si es público, cualquier persona podría acceder a esa información y usarla de forma malintencionada.

---

d) Escriban 3 cosas que aprendieron hoy que NO sabían antes:

- Cómo crear y trabajar con ramas de forma individual sin afectar el trabajo de los demás.
- Cómo resolver conflictos de fusión manualmente en Git.
- La importancia de usar `.gitignore` para proteger archivos sensibles y mantener limpio el repositorio.

Criterios de Evaluación

- Repositorio en GitHub con múltiples ramas fusionadas correctamente
- Cada integrante creó su propia rama, hizo commits y push desde su computador
- Al menos 1 conflicto provocado, documentado (captura) y resuelto exitosamente
- Archivo .gitignore configurado y funcionando (no se subieron .env ni .log)
- Uso correcto de Conventional Commits en los mensajes
- Cheat sheet completada con descripción propia de cada comando
- Calidad de las reflexiones grupales sobre buenas prácticas
- Participación activa de TODOS los integrantes (cada uno desde su computador)